EMBEDDED SYSTEMS

TASK-1

1. Introduction To Embedded System:

Embedded systems are specialized computing systems designed to perform specific tasks efficiently. They integrate hardware and software to operate independently or as part of a larger system. Unlike general-purpose computers (e.g., PCs), these systems are tailored for dedicated functions, often operating in real-time with limited resources like low power, small memory, and, in some cases, with little to no human intervention.

Classification Of Embedded System:

Embedded System can be classify based on two factors:

1.Performance and Functional Requirements
2.Performance of Micro-controllers

Based on Performance and Functional requirements, it is divided into 4 types:

I) Real-Time Embedded System: These are strictly time specific which means these embedded systems provides output within a defined time frame.These type of embedded systems provide quick response in critical situations which gives most priority to time based task performance and generation of output.

Eamples:

- Traffic control system
- Military usage in defence sector
- Medical usage in health sector

II) Stand Alone Embedded System: These are independent system which can work by themselves they don't depend on host system.It takes input in digital or analog form and provides output.

Examples:

- MP3 players
- Microwave ovens
- Calculator

III) Networked Embedded System: These systems are connected to a network which may be wired or wireless to provide output to attached device.They communicate with embedded web server through network.

Examples:

- Home security system
- ATM machine
- Card swipe machine

IV) Mobile Embedded System: These systems are small , portable and easy to use and requires less resources.They are the most preferred embedded systems.
Examples:

- Mobile phones
- Digital Camera
- MP3 player

Based on Performance of micro-controller, it is divided into 3 types as follows :
I) Small Scale Embedded Systems: These systems are designed using an 8-bit or 16-bit micro-controller. They can be powered by a battery. The processor uses very less/limited resources of memory and processing speed.

II) Medium Scale Embedded Systems: These systems are designed using an 16-bit or 32-bit micro-controller.Integration of hardware and software is complex in these systems.Java,C and C++ are the programming languages are used to develop medium scale embedded systems.

III) Sophisticated or Complex Embedded Systems: These systems are designed using multiple 32-bit or 64-bit micro-controller. These systems are developed to perform large scale complex functions. These systems have high hardware and software complexities.

Characteristics of Embedded systems:

- **Task-Specific Functionality**: Embedded systems are designed to perform a single, predefined task or a set of related tasks, ensuring optimized performance for their specific purpose.

- **Low Cost**: These systems are cost-effective due to their minimalistic design, making them suitable for mass production and widespread use.

- **Time-Specific Operations**: Embedded systems are often real-time, meaning they must complete tasks within strict time constraints to ensure proper functionality.

- **Low Power Consumption**: They are designed to operate with minimal power, making them ideal for battery-powered or energy-efficient applications.

- **High Efficiency**: Embedded systems are optimized for their specific tasks, ensuring high performance and reliability.

- **Minimal User Interface**: These systems often require little to no user interaction, simplifying their operation and usability.

- **Reduced Human Intervention**: Embedded systems are typically autonomous, requiring minimal human input to function effectively.

- **Stability and Reliability**: They are highly stable and reliable, consistently performing their tasks without frequent changes or failures.

- **Microcontroller or Microprocessor-Based**: Embedded systems use micro-controllers or microprocessors with limited memory and processing power tailored to their specific tasks.

- **Compact and Manufacturable**: Their small size and low complexity make them easy to manufacture and integrate into various devices.

These characteristics make embedded systems indispensable in applications ranging from consumer electronics and medical devices to industrial automation and aerospace systems. Their ability to deliver dedicated, efficient, and reliable performance underscores their importance in modern technology.

Real-World Applications:

- **Everyday Consumer Applications**

Embedded systems are widely used in **smartphones, tablets, and wearable devices** like smartwatches and fitness trackers, where they monitor health metrics, manage notifications, and enable wireless connectivity with other device. **Home appliances** such as washing machines, air conditioners, and digital cameras rely on embedded systems to control temperature, cycle timing, autofocus, and image stabilization.
Smart home devices like thermostats, lighting systems, and security cameras use embedded systems to automate and optimize energy usage and safety.

- **Automotive Industry**

Modern vehicles incorporate embedded systems for Engine Control Units **(ECUs)**, airbag deployment, and Advanced Driver Assistance Systems (ADAS) such as lanekeeping assist and adaptive cruise control. These systems ensure **realtime performance, safety, and fuel efficiency**, while also supporting infotainment and navigation features.

- **Healthcare and Medical Devices**

Embedded systems are critical in **medical equipment** for monitoring and regulating vital functions.Such devices provide precise, realtime data and treatment control. Their reliability and accuracy are essential for patient safety and effective healthcare delivery. Examples:
wearable health monitors, implantable pacemakers, glucose meters, and portable diagnostic tools.

- **Industrial Automation**

In manufacturing, embedded systems control **industrial robots, conveyor systems, and automated machinery**, enabling precise coordination, efficiency, and reduced human intervention. They are also used in **air conditioning and HVAC systems** to regulate temperature, airflow, and energy consumption based on environmental conditions.

- **Aerospace, Defense, and Transportation**

Embedded systems manage **drones, aircraft control systems, and traffic light management**, ensuring stability, navigation, andoptimized traffic flow. In

aerospace and defense, they provide **realtime monitoring, guidance, and control** for critical operations.

DIFFERENCE BETWEEN MICROCONTROLLER VS MICROPROCESSOR

Micro-controllers and microprocessors are both integrated circuits (IC), these single-chip processors are both extremely valuable in the continued development of computing technology. However, micro-controllers and microprocessors differ significantly in component structure, chip architecture, performance capabilities and applications.

A **microprocessor** is primarily a central processing unit (CPU) on a single chip. It requires external components like memory (RAM, ROM), input/output (I/O) ports, and timers to function
A **microcontroller**, on the other hand, is a compact system that integrates a CPU, memory (RAM, ROM, EEPROM), I/O ports, and peripherals like timers and analog-to-digital converters (ADCs) on a single chip.it do not require additional peripherals or complex operating systems to function, while microprocessors do.

Cost-effective and small-in-size, low-power micro-controllers are optimized for all-in-one functionality. As a result, these units are best used for specific applications like automotive infotainment systems and Internet-of-Things (IoT) devices.

Conversely, general-purpose microprocessors are typically more powerful and are designed to be supported by specialized hardware for increased performance in demanding applications like personal computing and graphics processing.

On a hardware level, microprocessors are based on the “classical” von Neumann architecture. This consists of a CPU with both an Arithmetic Logic Unit (ALU) and processor registers (small amounts of fast memory storage for quick data access), a control unit, memory for data and instructions, external memory for mass storage, and I/O mechanisms. This methodology uses the same set of interconnecting wires (known as a bus) to both transmit instructions and perform operations.It cannot perform these action Simultaneously.

On the other hand, micro-controllers use the more complex Harvard architecture, which has one dedicated set of data buses and address buses for reading and writing data to memory, and another set to fetch instructions for performing operations simultaneously.

the Harvard architecture, which separates program and data memory, enabling faster access and real-time performance. They are ideal for applications requiring low power consumption and compact design.
Where as the von Neumann architecture, where program and data memory share the same bus, which can lead to slower access but offers scalability and flexibility.
Microprocessors are powerful, capable of multitasking, and suited for High-Performance Computing(HPC) applications like gaming, graphics processing, and industrial computing.

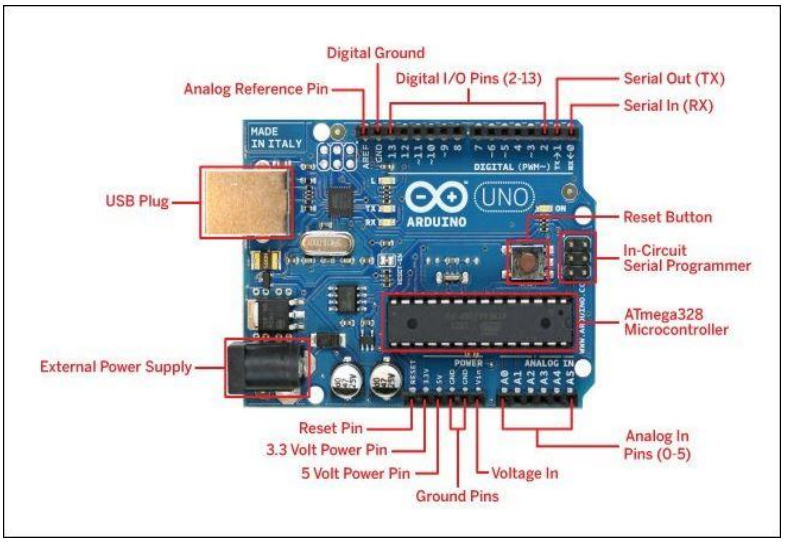
Key Differences:

Feature	Microcontroller	Microprocessor
Integration	CPU + Memory + I/O (single chip)	Only CPU
System Size	Compact	Requires external components
Power Consumption	Low	High
Performance	Moderate	High
Cost	Low	Higher
OS Support	Usually no OS / RTOS	Full OS
Application	Embedded systems	Computers, servers
Architecture	Harvard	Von Neumann
Examples	Arduino uno,ESP32	Intel core i5

2. COMPONENTS STUDYS

I) ARDUINO UNO

The Arduino Uno is an open-source micro-controller board based on the **ATmega328P** chip, widely considered the industry standard for learning electronics. It features 14 digital input/output (I/O) pins and 6 analog inputs, designed to interface easily with various sensors and actuators.



Pin Diagram Overview

The board is laid out with standard headers to simplify prototyping.

1. Digital I/O Pins (0–13)

These 14 pins are used for binary logic—either 5V (HIGH) or 0V (LOW).

- **Pins 0 (RX) & 1 (TX):** Dedicated to Serial Communication. Use these to send/receive data to a computer.
- **Pins 2 & 3:** Support External Interrupts, allowing the Arduino to stop its current task and respond immediately to an event like a button press.
- **PWM Pins (3, 5, 6, 9, 10, 11):** Marked with a tilde (~), these simulate analog output (e.g., dimming an LED) using Pulse Width Modulation.
- **Pin 13:** Connected to an onboard LED, perfect for "Blink" tests without extra wiring

2. Analog Input Pins (A0–A5)

These pins read continuous voltage levels between 0V and 5V.

- **Resolution:** They use a 10-bit Analog-to-Digital Converter **(ADC)**, converting the voltage into a value between 0 and 1023.
- **Secondary Functions:** A4 and A5 also serve as I2C communication lines (**SDA** and **SCL**), respectively.

3. Power and Special Pins

- **VIN:** Used to supply power (recommended 7–12V) when using an external source.
- **5V & 3.3V:** Regulated output pins to power external components.
- **GND:** Ground pins used to complete the circuit.
- **AREF:** The Analog Reference pin allows you to set an external reference voltage for analog readings.
- **RESET:** Pulling this pin LOW restarts the microcontroller's program.

Role of Arduino Uno:

1) Sensing (Data Acquisition)

- Interfaces with sensors via **ADC (analogRead)**, **I²C**, **SPI**, **UART**
- Converts real-world signals (temperature, light, motion) into digital data
- **Note:** 10-bit ADC on ATmega328P → 0–1023 quantization over reference voltage.

2) **Processing (Embedded Control Logic)**

- Executes firmware in a deterministic loop (setup() + loop())
- Implements **control algorithms** (thresholding, PID, filtering)

EXAMPLE:

if (temp > 35) digitalWrite(FAN, HIGH);

3) Actuation (Output Control)

- Drives **GPIO**, **PWM**, **timers** to control actuators
- Interfaces with relays, motors, LED's, buzzers

Key peripherals:

- PWM (~490/980 Hz) for motor speed / dimming
- Timers for precise delays and scheduling

4) Communication

- Serial (UART via USB), **I²C**, **SPI**
- With add-ons (shields), supports Wi-Fi/Bluetooth.

5) Rapid Prototyping & Ecosystem Leverage

- Massive library ecosystem (<DHT.h>, <Wire.h>, <Servo.h>)
- Plug-and-play **shields** (display, motor driver, GSM)
- Shortens development cycle from **weeks** → **hours**

Internal Architecture (ATmega328P)

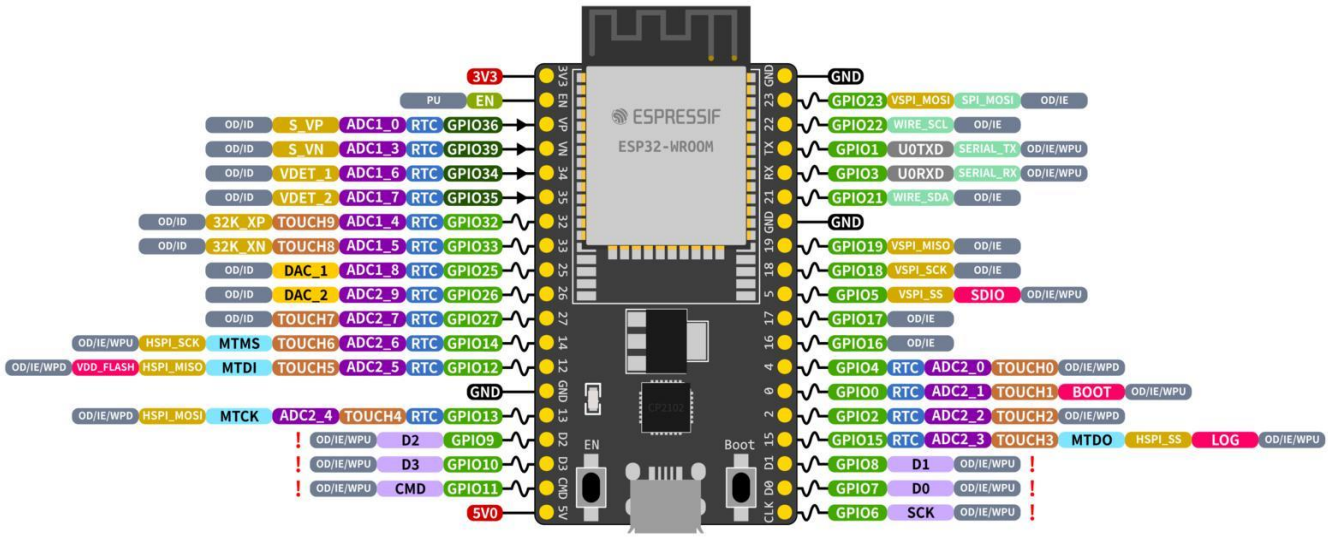
Block	Function
CPU (8-bit AVR)	Executes instructions (~16 MHz)
Flash (32 KB)	Stores program
SRAM (2 KB)	Runtime data
EEPROM (1 KB)	Non-volatile data
GPIO (14 digital, 6 analog)	I/O interfacing
Timers/Counters	Time control, PWM
ADC	Analog-to-digital conversion

Applications:

- **Prototyping & Education:** Due to its simple, replaceable microcontroller and open-source design, it is the standard for learning embedded electronics.
- **Robotics:** Used to control servos, motors, and sensors for autonomous or remote-controlled robots.
- **Home Automation (IoT):** Acts as a controller to manage lights, fans, and sensors (temperature, light, moisture) in smart home systems.
- **Sensor Monitoring Systems:** Reads analog data from sensors for logging environmental data, such as humidity or gas levels.
- **Industrial Automation:** Small-scale automation, such as traffic light controllers, parking lot counters, or packing machines.
- **Medical Instruments:** Used to create simple tools like heart rate monitors, thermometers, or digital weighing machines.
- **Wearable Electronics:** Used in specialized "LilyPad" or "Nano" versions for integrating electronics into clothing.

II) ESP32

The **ESP32** is a **highly integrated, dual-core microcontroller SoC** from Espressif, engineered for **connected (IoT) embedded systems**. It combines **compute + connectivity + rich peripherals** on a single chip, significantly reducing external components.



ESP32 Specs

32-bit Xtensa® dual-core @240MHz
Wi-Fi IEEE 802.11 b/g/n 2.4GHz
BLuetooth 4.2 BR/EDR and BLE
520 KB SRAM (16 KB for cache)
448 KB ROM
34 GPIOs, 4x SPI, 3x UART, 2x I2C,
2x I2S, RMT, LED PWM, 1 host SD/eMMC/SDIO,
1 slave SDIO/SPI, TWAI®, 12-bit ADC, Ethernet

PWM Capable Pin
GPIOX GPIO Input Only
GPIOX GPIO Input and Output
DAC_X Digital-to-Analog Converter
DEBUG JTAG for Debugging
FLASH External Flash Memory (SPI)
ADCX_CH Analog-to-Digital Converter
TOUCHX Touch Sensor Input Channel
OTHER Other Related Functions
SERIAL Serial for Debug/Programming
ARDUINO Arduino Related Functions
STRAP Strapping Pin Functions

RTC RTC Power Domain (VDD3P3_RTC)
GND Ground
PWD Power Rails (3V3 and 5V)
! Pin Shared with the Flash Memory
Can't be used as regular GPIO

GPIO STATE
WPU: *Weak Pull-up (Internal)*
WPD: *Weak Pull-down (Internal)*
PU: *Pull-up (External)*
IE: *Input Enable (After Reset)*
ID: *Input Disabled (After Reset)*
OE: *Output Enable (After Reset)*
OD: *Output Disabled (After Reset)*

Specifications:

- **Processor:** Dual-core Xtensa® 32-bit LX6 microprocessor, typically running at **240 MHz**.
- **Connectivity:** 2.4 GHz **Wi-Fi** (802.11 b/g/n) and **Bluetooth** 4.2 / BLE.
- **Memory:** 520 KB internal SRAM and 448 KB ROM. Most development boards include 4 MB of external Flash memory.
- **Peripherals:**

34 programmable GPIOs with multi-function support (PWM, I2C, SPI, UART).

18 ADC channels (12-bit) for precise analog sensor readings.

10 capacitive touch sensors and a built-in Hall effect sensor.

- **Security:** Hardware-accelerated encryption (AES, SHA-2, RSA) and secure boot features.
- **Power Management:** Ultra-low-power co-processor that allows the chip to consume as little as **5 µA in deep sleep mode**, making it ideal for battery-powered IoT devices.

ESP32 Variants

- **ESP32-S Series:** High-performance chips like the **S3**, which adds AI acceleration and USB support.
- **ESP32-C Series:** Cost-effective RISC-V based chips (e.g., **C3**) designed as a modern, pin-compatible replacement for the ESP8266.
- **ESP32-CAM:** A specialized board that integrates a small camera module and SD card slot for video streaming and image recognition.

Role of ESP32

1) Sensing (Data Acquisition)

- Interfaces with sensors via ADC (12-bit), I²C, SPI, UART, 1-Wire
- Supports multiple analog channels and capacitive touch inputs

Engineering detail: 12-bit ADC → 0–4095 resolution (higher granularity than typical 10-bit MCUs).

2) Processing (Control + Edge Compute)

- Dual-core Xtensa LX6 CPU (up to ~240 MHz)
- Executes concurrent tasks (e.g., sensor read + network stack)
- Supports FreeRTOS natively (task scheduling, semaphores, queues)

Implication: You can partition workloads—e.g., Core 0 handles Wi-Fi stack, Core 1 runs application logic.

3) Connectivity (Primary Differentiator)

- Integrated Wi-Fi (802.11 b/g/n)
- Bluetooth Classic + BLE
- Enables cloud integration, remote monitoring, OTA updates

Protocols commonly used: HTTP, MQTT, WebSockets.

4) Actuation (Output Control)

- Rich GPIO, PWM, DAC, timers
- Drives relays, motors, LEDs, buzzers with precise timing
- Hardware PWM channels (LEDC) allow fine-grained duty-cycle control.

5) Low-Power Operation

- Multiple sleep modes: light sleep, deep sleep (~μA range)
- Ideal for battery-powered IoT nodes

6) Security & Reliability

- Hardware features:
    - Secure boot
    - Flash encryption
    - Cryptographic accelerators
- Critical for **IoT deployments exposed to networks**

Internal Architecture:

Block	Function
Dual-core CPU	Parallel processing
SRAM (~520 KB)	Runtime operations
Flash (external)	Firmware storage
Wi-Fi/Bluetooth	Wireless connectivity
ADC/DAC	Analog interfacing
GPIO (~30 pins)	Digital I/O
Timers, PWM	Time-critical control

Applications

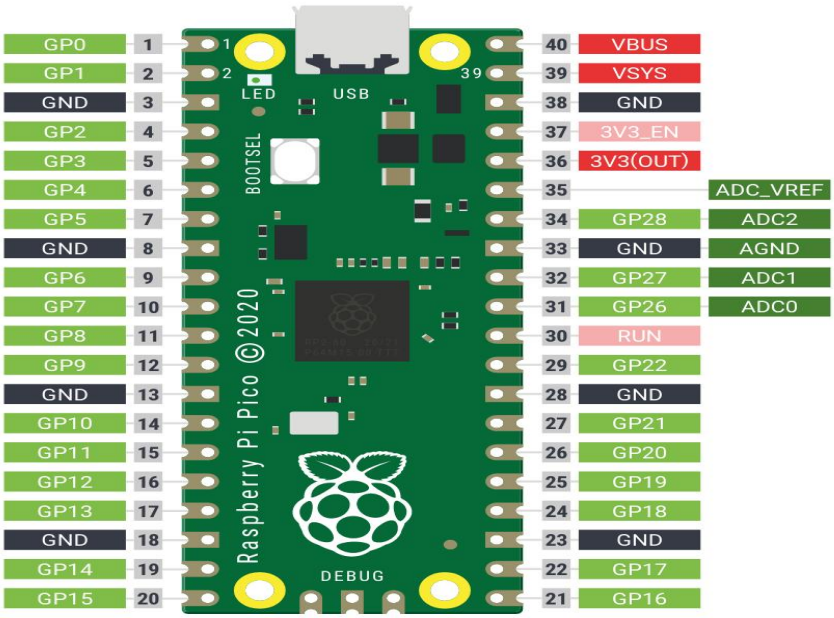
Because of its built-in wireless capabilities, the ESP32 is a staple in the **Internet of Things (IoT)** ecosystem:

- **Smart Home:** Controlling lights, thermostats, and smart locks via mobile apps.
- **Robotics:** Serving as the central controller for drones, autonomous rovers, and robotic arms.
- **Environmental Monitoring:** Collecting data from humidity, temperature, and air quality sensors to upload to the cloud.
- **Wearables:** Health monitors and fitness trackers that sync data via Bluetooth Low Energy

The ESP32 is highly accessible for developers as it can be programmed using various environments, most notably the Arduino IDE, MicroPython, and the official ESP-IDF.

Raspberry Pi Pico

The Raspberry Pi Pico is a low-cost, high-performance microcontroller board based on the **RP2040 chip**, designed for electronics projects and embedded systems. Unlike Raspberry Pi computers, it lacks an OS and runs single programs (C/C++ or MicroPython) instantly, featuring dual-core ARM processors, 264KB SRAM, and 40 GPIO pins for input/output.



Specifications:

- **Microcontroller:** Raspberry Pi RP2040 (dual-core Arm Cortex M0+ @ 133MHz).
- **Memory:** 264 KB SRAM and 16 MB onboard Flash memory.
- **GPIO:** 26/40 pins, including 3 analog inputs and 16 PWM channels.
- **Peripherals:** 2x UART, 2x SPI, 2x I2C, 16x PWM channels, 1x USB 1.1 controller and PHY.
- **Power:** 1.8V to 5.5V input; can be powered by USB or batteries.
- **Programmable I/O (PIO):** Allows the creation of custom hardware interfaces.

Role of Raspberry Pi Pico:

1) Sensing (Data Acquisition)

- Interfaces via ADC (12-bit internal, ~10–11 ENOB effective), I²C, SPI, UART
- Reads sensors (temperature, light, pressure)

Engineering note: Single ADC block multiplexed across inputs; careful sampling + averaging improves accuracy.

2) Processing (Deterministic Control)

- Dual-core ARM Cortex-M0+ (up to ~133 MHz)
- Executes bare-metal firmware or RTOS
- Predictable timing, low interrupt latency

Implication: Suitable for **real-time control loops** (motor control, signal timing).

3) Programmable I/O (PIO) — Key Differentiator

- Dedicated PIO state machines to implement custom protocols in hardware
- Offloads CPU for timing-critical I/O (e.g., WS2812 LEDs, precise pulse generation)
- Pico's standout role: hardware-assisted, cycle-accurate I/O orchestration.

4) Actuation (Output Control)

- GPIO **(26 usable pins)**, PWM slices, timers
- Drives LEDs, motors (via drivers), relays, buzzers

5) Communication

- Multiple UART, SPI, I²C controllers (flexibly mapped to pins)
- USB 1.1 device/host (for serial, HID, mass storage during flashing)

6) Low-Power, Cost-Efficient Design

- Low power draw relative to Wi-Fi MCUs
- Very low BOM cost → ideal for **education and scalable deployments**

RP2040 Architecture:

Block	Function
Dual-core Cortex-M0+	Application processing
SRAM (264 KB)	Program/data at runtime
External QSPI Flash	Firmware storage
PIO (8 state machines)	Custom, deterministic I/O
ADC	Analog sensing
PWM (8 slices)	Waveform generation
USB	Device/host connectivity

Applications:

- **Robotics and Automated Systems**

Raspberry Pi Pico can power robots and drones due to its small size and low power consumption. For example, it can control line-following robots, obstacle-avoidance robots, tricopters, or quadcopters using sensors like IR, ultrasonic, and accelerometers

- **Internet of Things (IoT)**

With the Pico W variant, IoT applications become possible. Projects include monitoring soil moisture for plants, sending alerts via text, or displaying environmental data on IoT dashboards in the cloud

- **Education and Learning Programming**

Raspberry Pi Pico is beginner friendly and widely used to learn MicroPython or C programming.
Examples include programming a simple blinking LED, building a small weather station, or creating interactive games.

- **Home Automation and Control**

Projects include smart door locks with biometric or keypad access, coffee grinders with automated control, or multifunctional control knobs for computer applications

- **Advanced Projects and Experimentation**

Advanced users use Pico in robotic arms (6 DOF), cryptocurrency ticker monitors, or realtime Mandelbrot visualization on OLED displays.Its programmable I/O (PIO) allows handling complex tasks in parallel, such as motor control while reading multiple sensors.

Basic Seansors

Temperature sensor

A **temperature sensor** is an instrument used to measure the degree of hotness or coldness of a physical object, environment, or system and transform this information into a form that can be monitored, recorded, or used for processing (e.g., electrical signals such as voltage, current, or resistance) by a microcontroller or system.

Major Types of Temperature Sensors:

1. **Thermocouples:** Thermocouples are made of two dissimilar metals joined at one end. They generate a voltage proportional to the temperature difference between the ends. They are versatile and used in a wide temperature range.

Characteristics:

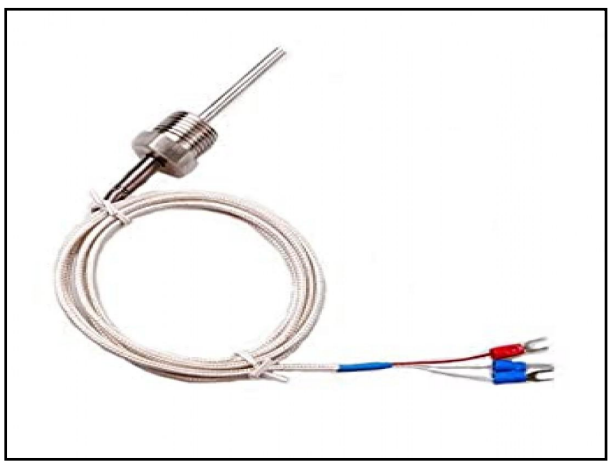
- Very wide temperature range (up to >1000°C)
- Generates small voltage (μV range)
- Requires amplification

Use case: Furnaces, engines, heavy industry

2. **Resistance Temperature Detectors (RTDs):** RTDs use the fact that the electrical resistance of certain materials, such as platinum, changes linearly with temperature. They offer high accuracy and stability.

Example

- PT100 (100 Ω at 0°C)



Characteristics:

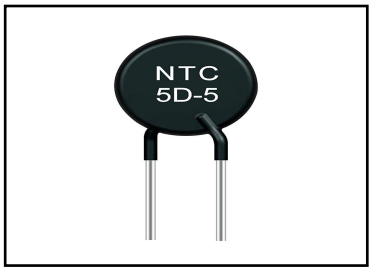
- High accuracy and stability
- Nearly linear
- More expensive

Use case: Industrial measurement systems

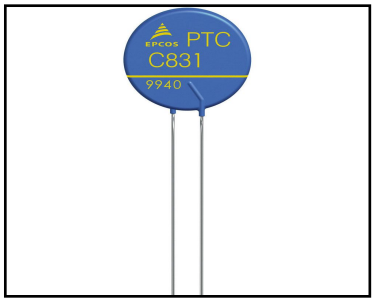
3. **Thermistors:** Thermistors are temperature-sensitive resistors. They exhibit a significant change in resistance with temperature, making them useful for high-accuracy measurements.

Types

- **NTC (Negative Temp Coefficient)** → Resistance ↓ as temp ↑



- **PTC (Positive Temp Coefficient)** → Resistance ↑ as temp ↑



Characteristics

- Highly sensitive
- Non-linear (needs calibration/lookup table)

Use case: Battery packs, chargers, HVAC

.

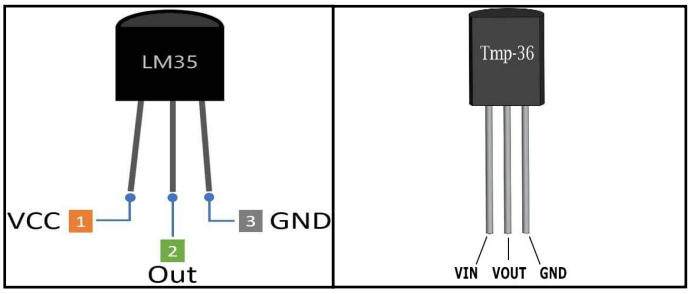
4. Integrated circuit Temperature Sensors(Analog & Digital):

IC temperature sensors integrate the **sensing element + signal conditioning + (often) ADC + digital interface** on a single chip. They provide **calibrated, stable temperature readings** with minimal external circuitry—ideal for embedded systems.

Analog IC sensors:

Examples

- LM35 temperature sensor
- TMP36



Characteristics:

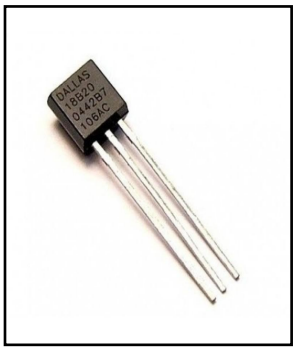
- Output: **Linear analog voltage**
- Simple interfacing (ADC required)
- Fast response

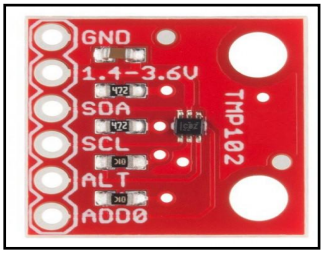
Trade-off: Susceptible to **noise** and ADC accuracy limits

Digital IC sensor:

Examples

- DS18B20 (1-Wire)
- TMP102 (I^2C)





Characteristics

- Output: **Direct digital temperature**
- High accuracy (±0.5°C typical)
- Supports **multi-device bus** (e.g., DS18B20 unique IDs)

Advantage: Immune to analog noise, easier scaling

Applications:

Temperature sensors are used to measure the temperature of various objects or environments. They are commonly used in a wide range of applications, including:

1. **Domestic equipment:** Temperature sensors are used in thermometers, refrigerators, water heaters, air-conditioners, and other household appliances to monitor and control temperature.

2. **Industrial processes:** Temperature sensors are used in industrial processes to monitor and control temperature in manufacturing, chemical processing, food processing, and other industries.

3. **Building and structure monitoring:** Temperature sensors are used to monitor the temperature of structures, buildings, and infrastructure to ensure safety and structural integrity.

4. **Environmental monitoring:** Temperature sensors are used in environmental monitoring systems to measure the temperature of soil, water bodies, and air to study climate change, weather patterns, and ecological systems.

5. **Medical applications:** Temperature sensors are used in medical devices such as thermometers, incubators, and patient monitoring systems to measure body temperature and ensure proper medical care.

6. **Research and development:** Temperature sensors are used in scientific research and development to study thermal properties, conduct experiments, and gather data for analysis.

Motion Sensor

A **motion sensor** is an electronic device that **detects movement of objects or people** within a defined area and converts that event into an **electrical signal** (digital HIGH/LOW or analog), which a controller can process.

The effectiveness of motion sensors hinges on their ability to accurately detect genuine movement while minimizing false triggers. Advanced models can distinguish between different types of motion, allowing them to be more selective in the actions they trigger. As technology evolves, the precision and utility of motion sensors continue to improve, making them a staple in the toolkit of modern security and automation systems.

Major Types of Motion Sensors:

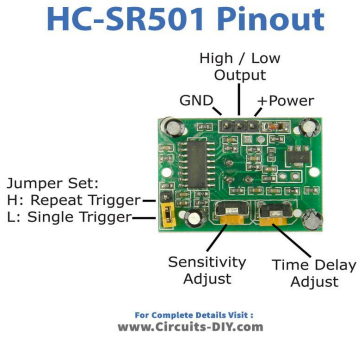
1. PIR Motion Sensor:

PIR sensor is the pyroelectric sensor component, which is sensitive to infrared light. When an object with a different temperature from the surroundings enters the sensor's detection range, it disrupts the existing infrared energy pattern. This

disturbance causes the sensor to react, usually resulting in the activation of a connected device, such as a light or alarm system.

Example

- HC-SR501 PIR motion sensor



Characteristics

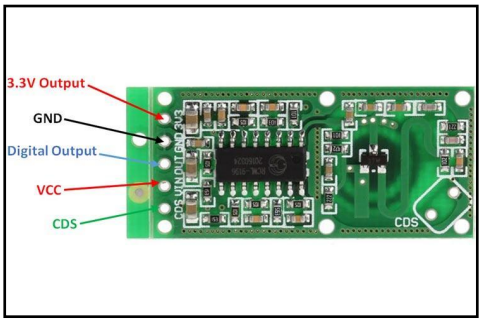
- Output: **Digital (HIGH on motion)**
- Range: ~3–7 m
- Low power, low cost
- Adjustable **sensitivity** and **trigger time**

2. Microwave(Radar) Sensor:

MW sensors involves a continuous wave of microwave radiation. When these waves encounter a moving object, they bounce back to the sensor with a different frequency due to the Doppler effect. This change in frequency indicates movement, triggering the sensor to activate an alarm, light, or other connected systems.

Example

- RCWL-0516 microwave radar sensor



Characteristics

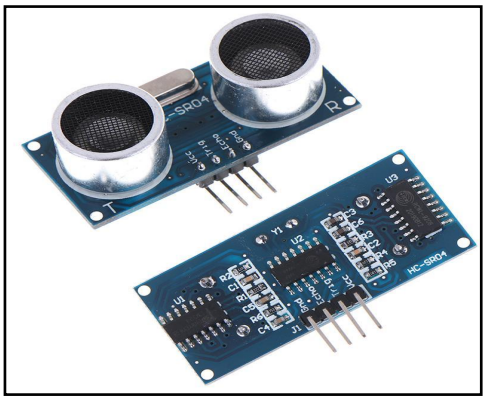
- Detects micro-movements
- Works through plastic, glass
- More prone to environmental noise

3. Ultrasonic Sensors:

Ultrasonic sensors emit high-frequency sound waves and analyze the echo received after it bounces off objects. Movement is detected when the returning sound waves change in frequency, a phenomenon known as the Doppler shift.

Example

- HC-SR04 ultrasonic sensor



Characteristics

- Measures distance; motion inferred by **Δdistance/Δtime.**
- Requires periodic triggering

Applications:

- **Home Security:** One of the most common applications for motion sensors is in home security. They are used to detect unauthorized entry, triggering alarms and notifying homeowners or security services. These sensors can also be connected to cameras to start recording when motion is detected, providing real-time surveillance.

- **Home Security:** One of the most common applications for motion sensors is in home security. They are used to detect unauthorized entry, triggering alarms and notifying homeowners or security services. These sensors can also be connected to cameras to start recording when motion is detected, providing real-time surveillance.

- **HVAC Efficiency:** In heating, ventilation, and air conditioning (HVAC) systems, motion sensors help in managing the climate within buildings efficiently. They adjust the temperature based on the occupancy of rooms, reducing the energy used in heating or cooling empty spaces.
- **Healthcare:** In healthcare facilities, motion sensors assist in patient monitoring without invasive supervision. They can alert staff if a patient falls or behaves unusually, enhancing patient safety and care.

- **Industrial Automation:** In industrial settings, motion sensors are part of automated systems that improve safety and efficiency. They can detect the presence of personnel in dangerous areas or automate processes by starting and stopping machinery based on worker presence.

Light sensor

A **light sensor** (optical sensor) is a device that **detects the intensity of light** (visible/IR/UV) and converts it into an **electrical signal**—typically a **change in resistance, current, voltage, or a digital lux value**—for processing by a controller.

Operating principles:

1) Photoconductive Effect (LDR)

- Incident light **reduces resistance** of a semiconductor material
- Dark → high resistance; Bright → low resistance

2) Photoelectric Effect (Photodiode/Phototransistor)

- Light generates **charge carriers** → **current/voltage**
- Often used in **reverse bias** for linear response

3) Integrated Optical Sensing (Digital ICs)

- On-chip photodiodes + ADC + calibration
- Output directly in **lux (illuminance)** via digital interface

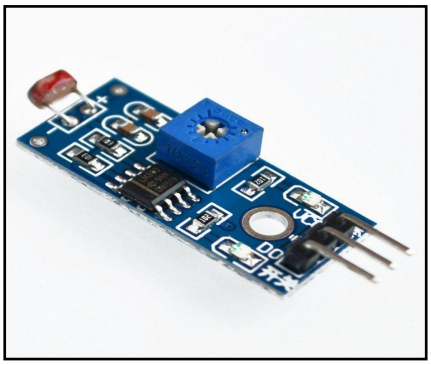
Major Types of Light Sensors:

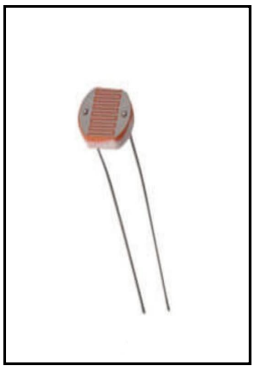
1. Photoresistors (LDRs)

Photoresistors, also known as light-dependent resistors (LDRs), are passive components whose resistance decreases with increasing light intensity. They are made of semiconductor materials like cadmium sulfide (CdS) or gallium arsenide (GaAs). LDRs are inexpensive, easy to use, and suitable for applications where precise light measurement is not critical.

Examples:

LM393





Characteristics

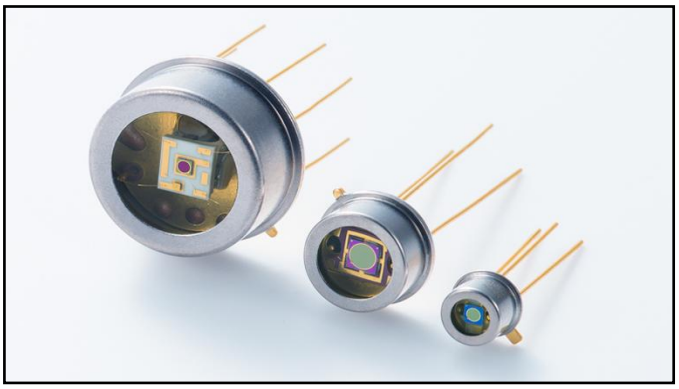
- **Analog**, non-linear
- Very low cost, easy to use (voltage divider)
- Slow response (tens of ms)

Typical use: automatic street lights, day/night switches

2. Photodiodes

Photodiodes are semiconductor devices that generate a current or voltage when exposed to light. They have a p-n junction that allows current to flow in one direction when light strikes the device. Photodiodes are fast, sensitive, and have a linear response to light intensity. They are commonly used in applications requiring precise light measurement and fast response times.

Examples:



Characteristics

- Fast response (µs–ns)
- Good linearity

- Requires amplification (transimpedance amplifier)

Typical use: optical communication, precision sensing

3. Phototransistors

Phototransistors are similar to photodiodes but with an additional amplification stage. They consist of a photodiode and a transistor, where the photodiode's current controls the transistor's base current. Phototransistors offer high sensitivity and current gain, making them suitable for low-light applications and scenarios requiring amplification.

Example:

Characteristics

- Higher sensitivity than photodiode
- Slower than photodiode
- Outputs amplified current

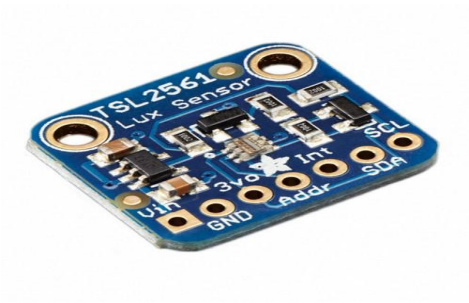
Typical use: light-triggered switches, counters

4. Ambient Light Sensors (ALS)/ Lux Sensor

Ambient light sensors (ALS) are designed to measure the overall illumination level in an environment. They often use a combination of photodiodes with different spectral sensitivities to mimic the human eye's perception of brightness. ALS are commonly found in smartphones, tablets, and laptops to automatically adjust screen brightness based on ambient light conditions.

Examples

- BH1750 light sensor
- TSL2561



Characteristics

- Direct lux output (digital)
- High accuracy and calibrated
- Easy I^2C interface

Typical use: smartphones, display auto-brightness, IoT

Applications:

Applications of Light Sensors Light sensors are versatile and widely used across diverse fields:

- Consumer Electronics Smartphones: Auto-brightness and camera metering. Laptops and TVs: Screen dimming and energy saving.
- Smart Lighting Systems Automatic light adjustment based on ambient brightness. Used in homes, offices, and public buildings.
- Agriculture Measurement of sunlight for photosynthesis optimization. Spectral light analysis for crop health.
- Solar Energy Track solar irradiance for panel optimization. Used in solar tracking systems.
- Industrial Automation Light sensors as part of machine vision systems. Conveyor belt sorting and counting applications.
- Security and Safety Used in motion detectors and intruder detection. Night/day detection for CCTV cameras.
- Automotive Industry Automatic headlight switching. Cabin light adjustment based on ambient lighting.
- Environmental Monitoring Monitoring UV levels for climate and ozone studies. Light pollution analysis in cities.

MINI-PROJECT

Smart Temperature Monitor

Working Principle

The Smart Temperature Monitoring System operates by continuously sensing and evaluating temperature in real time using a digital temperature sensor. The **DS18B20 sensor** measures the ambient or object temperature and converts it directly into a digital signal using the OneWire communication protocol.

This digital temperature data is transmitted to the **ESP32 microcontroller**, which processes the value without the need for analog-to-digital conversion. The ESP32 continuously compares the measured temperature with a predefined threshold value (for example, 40°C).

If the temperature exceeds this threshold, the microcontroller activates output devices such as an **LED and buzzer**, providing both visual and audible alerts. If the temperature remains within the safe range, the system stays in a normal state with no alert.

The system operates in a continuous loop, enabling real-time monitoring and immediate response to temperature changes. Additionally, due to the built-in Wi-Fi capability of the ESP32, the system can be further enhanced to transmit temperature data to cloud platforms for remote monitoring and alert notifications, making it suitable for IoT-based applications.

Circuit Design:

DS18B20

- VCC → 3.3V
- GND → GND
- DATA → GPIO4
- 4.7kΩ resistor → between DATA and VCC

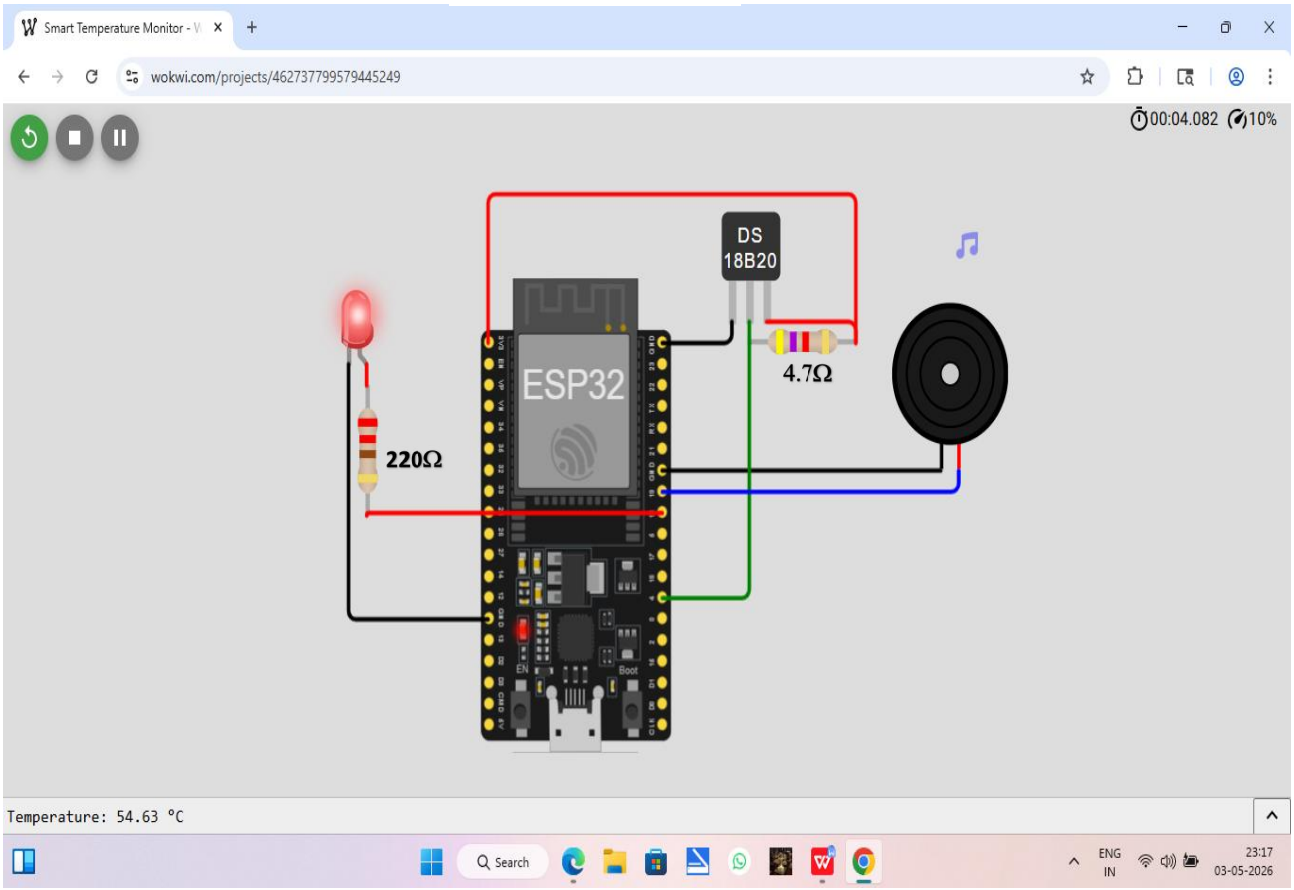
LED

- GPIO18 → 220Ω resistor → LED → GND

Buzzer

- GPIO19 → buzzer → GND

Screenshot



Code:

```
#include <OneWire.h>
#include <DallasTemperature.h>

#define ONE_WIRE_BUS 4

OneWire oneWire(ONE_WIRE_BUS);
DallasTemperature sensors(&oneWire);

int ledPin = 18;
int buzzerPin = 19;

// Thresholds (with hysteresis for stability)
float thresholdHigh = 40.0;
float thresholdLow  = 38.0;

bool alertState = false;

// Beep timing (milliseconds)
unsigned long previousMillis = 0;
const long beepOnTime  = 1000;   // buzzer ON duration
const long beepOffTime = 300;   // buzzer OFF duration
```

```
bool buzzerState = false;

void setup() {
  Serial.begin(115200);
  sensors.begin();

  pinMode(ledPin, OUTPUT);
  pinMode(buzzerPin, OUTPUT);

  Serial.println("Smart Temperature Monitor with Beep Pattern");
}

void loop() {
  // Read temperature
  sensors.requestTemperatures();
  float temp = sensors.getTempCByIndex(0);

  Serial.print("Temperature: ");
  Serial.print(temp);
  Serial.println(" °C");

  // Hysteresis logic
  if (temp >= thresholdHigh) {
    alertState = true;
  } else if (temp < thresholdLow) {
    alertState = false;
  }

  // LED follows alert state
  digitalWrite(ledPin, alertState);

  // Buzzer pattern (non-blocking)
  unsigned long currentMillis = millis();

  if (alertState) {
    if (buzzerState && (currentMillis - previousMillis >=
beepOnTime)) {
      buzzerState = false;
      previousMillis = currentMillis;
      digitalWrite(buzzerPin, LOW);
    }
    else if (!buzzerState && (currentMillis - previousMillis >=
beepOffTime)) {
      buzzerState = true;
      previousMillis = currentMillis;
      digitalWrite(buzzerPin, HIGH);
    }
  } else {
```

```
    // Turn OFF buzzer when safe
    digitalWrite(buzzerPin, LOW);
    buzzerState = false;
  }

  delay(500); // small delay for readability
}
```

Simulation link: https://wokwi.com/projects/462737799579445249

Upgradable:

"The ESP32 has integrated Wi-Fi, so it can transmit sensor data using HTTP or MQTT protocols to cloud platforms like Blynk or ThingSpeak. The temperature data is uploaded in real-time, and if it exceeds a threshold, alerts can be sent to the user's smartphone. This converts the system from a standalone embedded system into an IoT-enabled monitoring system.By integrating Wi-Fi, the system evolves from a local monitoring device to a smart IoT system capable of real-time remote supervision and intelligent decision-making."

Applications:

- **Industrial Equipment Protection**

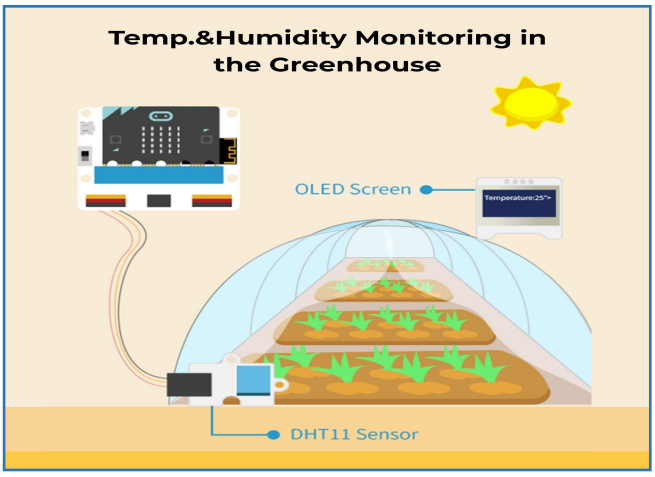
A smart temperature monitoring system is widely used in industrial environments to prevent overheating of machinery such as motors, transformers, and generators. The

DS18B20 sensor continuously measures the temperature of the equipment, and the ESP32 processes this data in real time. When the temperature exceeds a predefined threshold, the system activates an alert through an LED and buzzer, and can also be extended to shut down the machine using a relay. This helps in **preventing equipment damage, reducing maintenance costs, and ensuring operational safety.**

- ## Medical Cold Chain Monitoring

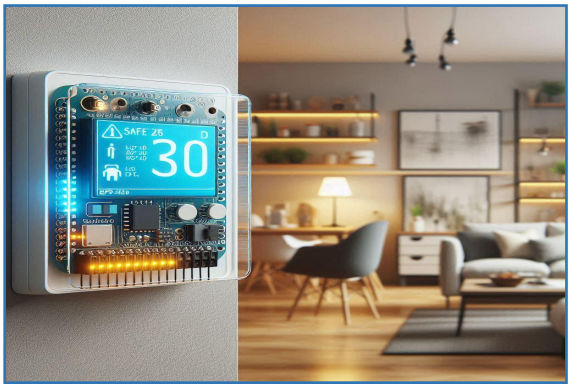
In healthcare applications, maintaining proper temperature is critical for storing vaccines, blood, and medicines. The smart temperature monitoring system ensures that storage conditions remain within the required range. The DS18B20 sensor continuously monitors the temperature, and the ESP32 compares it with safe limits. If any deviation occurs, the system generates an alert and can send notifications using IoT technology. This ensures the **effectiveness of medical products**, **prevents spoilage**, and **supports compliance** with health regulations.

- ## Smart Agriculture / Greenhouse



In agriculture, temperature plays a vital role in crop growth and productivity. The smart temperature monitoring system can be used in greenhouses to maintain optimal environmental conditions. The DS18B20 sensor measures the temperature, and the ESP32 automatically controls devices such as fans or irrigation systems when the temperature crosses a threshold. This reduces manual effort, **improves crop yield**, and **enables efficient resource managemen**t. With IoT integration, farmers can monitor conditions remotely.

- ## Home Fire Early Warning System



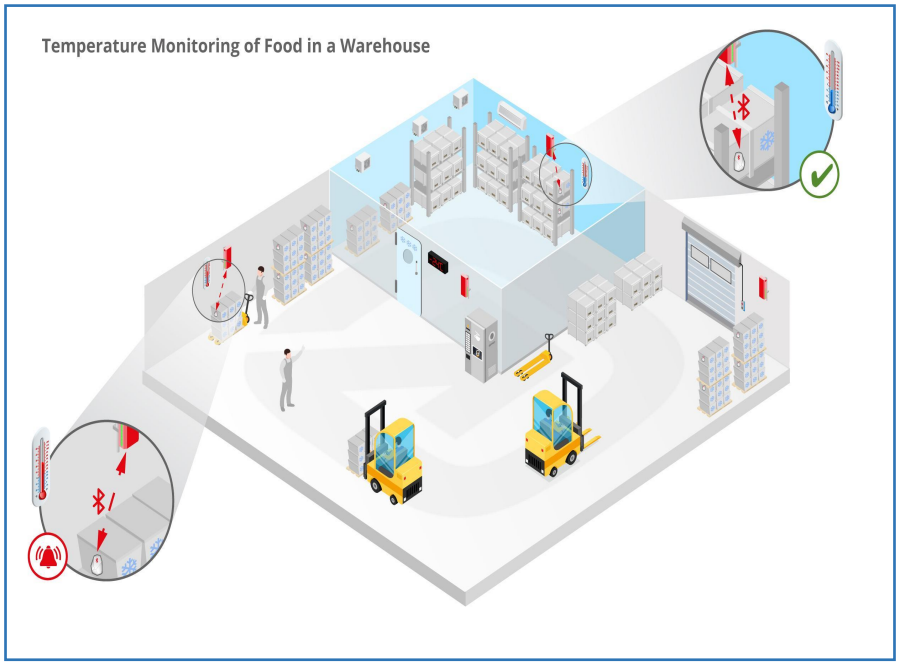
The system can be applied in residential environments as an early fire detection mechanism. A sudden rise in temperature can indicate the presence of fire. The DS18B20 sensor detects this increase, and the ESP32 immediately activates a buzzer and LED to alert occupants. This early warning helps in taking quick action to **prevent damage and ensures safety**. The system can be further enhanced with IoT to send alerts to mobile devices.

- ## Data Center Monitoring

In data centers and server rooms, maintaining optimal temperature is crucial to prevent overheating of electronic equipment. The smart temperature monitoring system uses the DS18B20 sensor to measure ambient temperature and the ESP32 to process the data. When the temperature rises beyond safe limits, the system activates

alerts and can be integrated with cooling systems. This ensures **system reliability, prevents hardware failure, and maintains continuous operation**.

- ## Food Storage and Cold Chain



In food storage and transportation, maintaining proper temperature is essential to preserve quality and safety. The smart temperature monitoring system continuously tracks the temperature of storage units and refrigerated vehicles. If the temperature exceeds safe limits, the ESP32 triggers an alert, helping to prevent spoilage. With IoT integration, temperature data can be logged and monitored remotely, **ensuring compliance with food safety standards and reducing losses**.